

Python / Pandas Analysis — Explanation

Day 4 of 7 — perfume_analysis.ipynb & perfume_analysis.py

What's in This Folder

- **perfume_analysis.ipynb** — the full notebook, already executed, with charts and outputs baked in so you can open it and see results immediately.
- **perfume_analysis.py** — the same analysis as a plain script (useful for running from the command line or a portfolio repo where notebooks aren't ideal).
- **charts/** — 6 PNG charts exported at 150 DPI, ready to drop into a resume, LinkedIn post, or slide deck.

Libraries Used

Library	Purpose
pandas	Load, clean, group, and aggregate the 3,000-row dataset
numpy	Vectorized calculations (e.g., repeat-customer flagging)
matplotlib	All 6 charts — line, horizontal bar, scatter, stacked bar, 2 pies

Analysis Walkthrough

The CSV is loaded with Order_Date parsed as a real datetime. The script then checks shape, dtypes, missing values, and duplicate Order_IDs — the same first-five-minutes checklist any analyst should run before trusting a dataset.

2. Feature Engineering

Two derived columns are added: Brand_Tier (Premium / Mid / Budget, based on the 12 brands) and Month (for time-series grouping). Neither exists in the raw CSV — they're created in Python to make the downstream groupby operations cleaner.

3. Six Charts, Six Questions

- **Monthly Revenue Trend** — is there real seasonality, or is that assumption wrong?
- **Revenue by City** — which cities should get more marketing budget?
- **Brand Revenue vs. Volume** — does the premium-vs-budget trade-off actually hold?
- **Delivery Status by Brand Tier** — do premium perfumes really return more often?
- **Payment Method Mix** — is UPI genuinely dominant?
- **Repeat vs. One-Time Revenue Share** — how much does customer loyalty matter?

Key Metrics Produced

Total Revenue: Rs. 2,00,27,271 | Total Orders: 3,000 | Average Order Value: Rs. 6,676 | Unique Customers: 723 | Return Rate: 9.1% | Cancellation Rate: 5.6% | Top City: Bengaluru | Top Brand: Dior

Interview Talking Points

- Why pandas groupby + agg() instead of manual loops — vectorized operations scale to millions of rows where Python loops would not.
- Why a derived Brand_Tier column instead of hardcoding brand names into every chart — it keeps the analysis DRY and easy to re-run if brands are added later.
- Why matplotlib was used directly instead of seaborn — full control over styling to match the brand palette used across the whole portfolio (Excel, Tableau, PDF).
- How you'd productionize this: wrap the script in a scheduled job (e.g., Airflow) that re-runs monthly and re-exports the charts and CSV summaries automatically.